HW3 — Before/After Experiment Comparison

ReWear AI | User Story 5: Wardrobe Search and Filtering

Experiment date: October 29, 2026 • Follow-up verification: October 30, 2026

1. Objective and controlled setup

The experiment evaluated whether project-specific context improved AI-assisted implementation of the same wardrobe search-and-filter feature. Both runs began from copies of the same Next.js starter application containing 12 sample clothing items, without search or filtering.

Condition	Run A — without rules	Run B — with rules and PRD
Model / tool	Claude Code, Opus 5.5 Medium	Claude Code, Opus 5.5 Medium
Initial code	Same 12-item Next.js baseline	Same 12-item Next.js baseline
Additional context	No AGENTS.md or PRD supplied	150-line AGENTS.md and docs/PRD.md read first
Feature request	Identical prompt: search and filter by category, color, season and formality; combine and clear filters	Identical feature prompt after separate context-loading message

Method: Each run was isolated in a separate folder and fresh Claude Code conversation. The original feature request was held constant; Run B additionally read the rules and PRD before implementation. Both implementations were left unchanged for the initial evaluation.

2. Measured outcomes

Measure	Run A	Run B
Manual browser checks	5/5 passed (user-confirmed)	5/5 passed (user-confirmed)
Persistent automated tests	0 created	12 created; 12/12 passed with Node.js test runner
Test-first sequence	Not demonstrated	Test written and failed before service was implemented
Production build / lint	Passed / passed	Passed / passed
Filtering architecture	Pure function in lib/; client browser component	Pure service in services/; browser component and separate ClothingCard
Empty wardrobe state	No dedicated empty-wardrobe state	Dedicated empty-wardrobe state
No-match result	Message shown	Message plus clear-filters action

The five shared browser checks were: (1) search “white” → two matching items; (2) category “Top” → three items; (3) Top + Casual → two items; (4) “purple unicorn” → zero results and an empty-state message; (5) clear filters → all 12 items. Both runs passed, as reported after manual testing.

3. Specific differences observed

Automated testing and TDD

Run A produced functional filtering code and performed ad hoc checks, but did not add a reusable test file. Run B wrote services/filterClothing.test.ts before implementing the service, recorded an expected missing-module failure, implemented the service, corrected one mistaken test expectation, and completed 12 passing tests across three suites using Node.js's built-in test runner (node --test). This was a departure from the project's planned Vitest tooling, but it provided executable, persistent unit tests without installing a new dependency.

Architecture and interface behavior

Both runs separated filter logic from UI state. Run B went further by extracting ClothingCard.tsx, generating filter options from the dataset, labeling the search control visibly, and providing an explicit empty-wardrobe state and a recovery action for no results. Run A already had responsive cards, labeled dropdowns, a result count, and a no-match message, so it would be inaccurate to describe its implementation as poor.

One behavior that differed, without an agreed winner

Run A treated "All-season" items as matches when a user selected a specific season (for example, Winter). Run B matched season values exactly. The frozen user story did not settle this interpretation, and the five common browser checks did not exercise it. This should be resolved in product requirements before the final P2 implementation, not counted automatically as a Run B improvement.

4. Interpretation and limitations

- Functionality: no measured improvement in the shared manual pass rate (5/5 in both runs).
- Development discipline: Run B added 12 persistent automated tests and documented a test-first sequence; Run A did not create a reusable test suite.
- Causality: Run B received AGENTS.md and the PRD together, plus a separate context-loading turn. The experiment cannot isolate the rules file from the PRD or from the extra preparation step.
- Scope: both runs used local sample data, not MongoDB, authentication, APIs, or a production deployment; server-side access control was therefore not evaluated.
- Verification: Run B's tests covered the filtering service, not a complete end-to-end browser flow. A fresh Node.js test run on October 30 confirmed 12 passing tests and no failures. The isolated folders did not have Git history to independently establish a test-first commit sequence.
- Evidence: screenshots show Run A's interface, Run B's context setup, Run B's unfiltered wardrobe and no-match state, and a successful Run B Node.js test-run summary. The shared five manual browser checks were confirmed by the user; the screenshots do not independently document every check.

5. Evidence index and next refinement

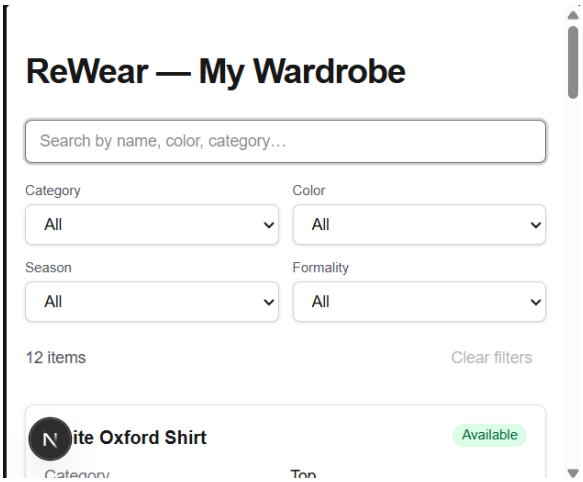
Evidence reviewed: the Run A and Run B implementation records; browser screenshots of Run A's wardrobe and a filtered result; Run B's project-instructions workspace, full wardrobe, and no-results state; a fresh Run B automated-test result; and the five manual checks confirmed for both runs.

Proposed rules refinement (not yet applied): clarify whether "All-season" should match a selected season, and specify a minimum test matrix for User Story 5 (search, each filter, combinations, reset, no results, empty wardrobe). Keep this as a proposed change until the team reviews it; do not present it as already committed.

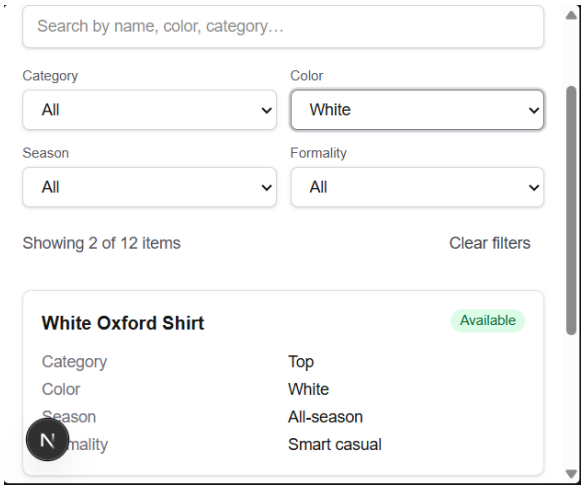
Appendix — Available visual evidence

The screenshots below show the interface states and test evidence available for each run. They do not document every manual test or imply that unpictured states were independently verified.

A1. Run A interface and filtered state

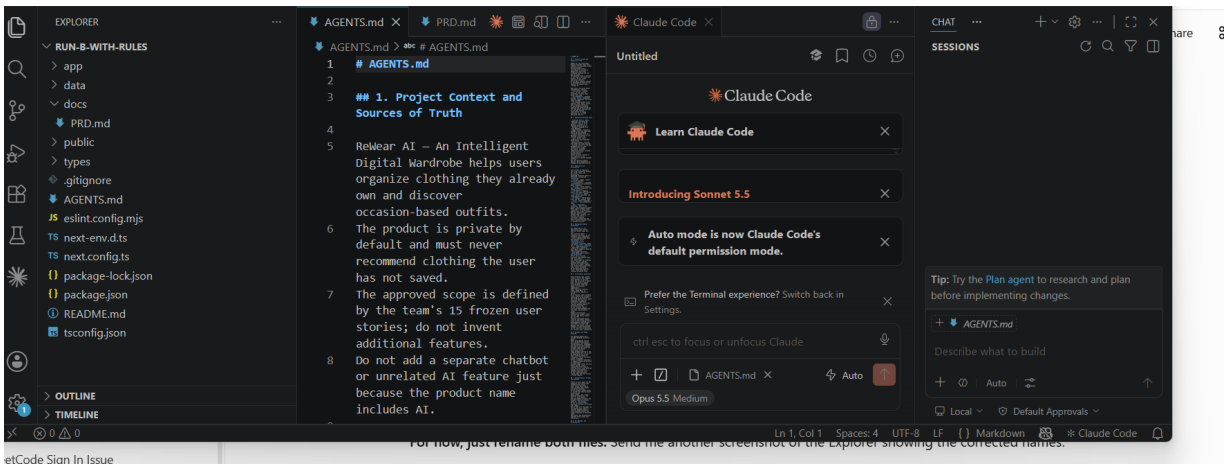


Run A: search and four filter controls; 12 items before filtering.



Run A: Color = White; “Showing 2 of 12 items.”

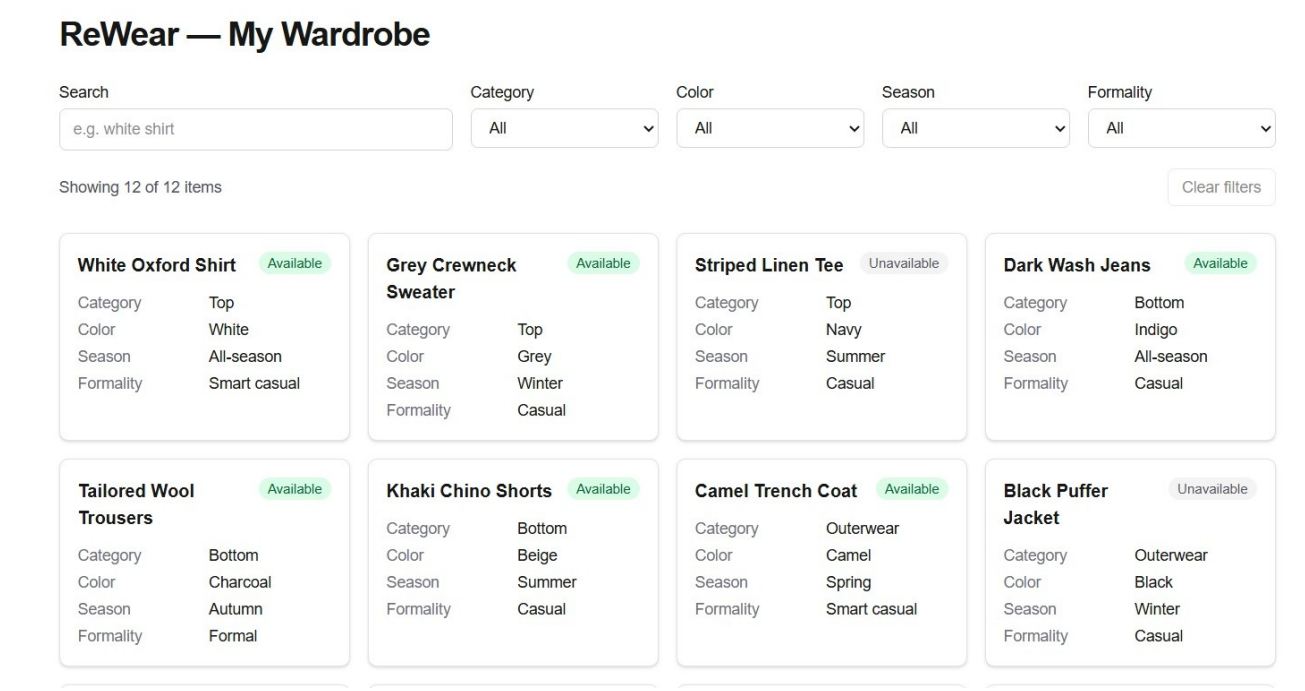
A2. Run B context files present before implementation



Run B workspace showing the project instructions (AGENTS.md) and product requirements (docs/PRD.md). The implementation record confirms these documents were read before the feature request.

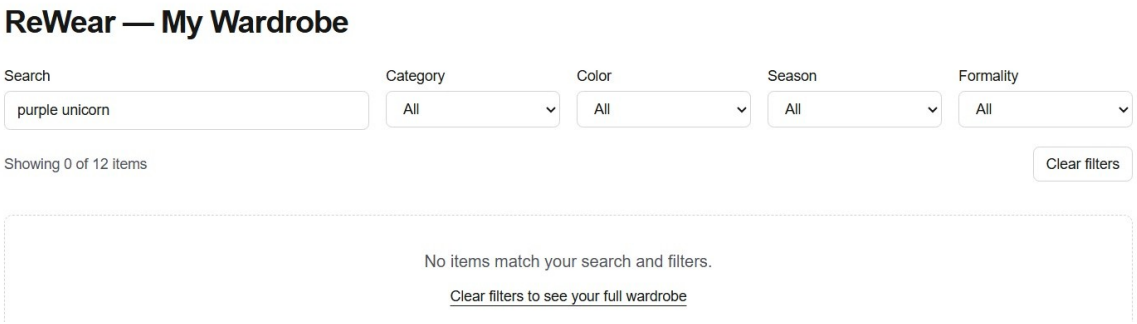
A3. Run B - unfiltered wardrobe

Run B displays the search control, four filter dropdowns, and “Showing 12 of 12 items” before filtering.



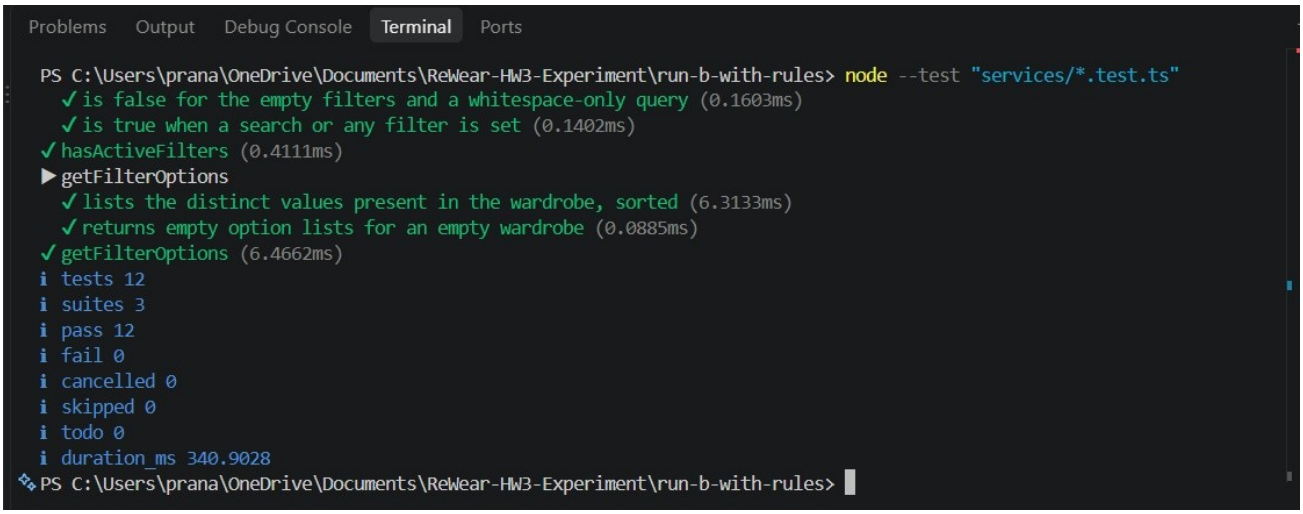
A4. Run B - no-match state and recovery

Search query “purple unicorn” yields 0 of 12 items, an explicit no-results message, and a clear-filters recovery action.



A5. Run B - automated tests (fresh verification)

The October 30 PowerShell run used node --test "services/*.test.ts". The terminal shows 12 tests, 3 suites, 12 passes, and 0 failures.



Verification note

The automated-test screenshot shows a fresh run using Node.js's built-in test runner, matching the test framework used during Run B. All 12 tests passed without changes to the application code.

Bottom line

Both runs met the same five manual checks. The strongest observed benefit of additional context was not a higher manual pass rate, but persistent automated tests, a recorded test-first sequence, and more explicit handling of empty and no-match states. The experiment does not establish which of the additional instructions caused these differences.